DATA ENGINEERING PORTFOLIO PROJECT REPORT

Enterprise-Grade Full-Stack Hosting, Obfuscation Pipelines, and Google Cloud Platform (GCP) Architecture

Prepared For: Public Presentation & Industry Review
Engineer: Archit Somayajula
Core Specialty: Data Engineering & Platform Architecture
Deployment Targets: GCP App Engine, Firebase Hosting, SQLite Analytics
Date: June 9, 2026

1. Executive Summary & Goals

This project documentation details the engineering process, codebase architectural layout, and full-stack security implementations of Archit Somayajula's Data Engineering Portfolio. The primary goal of this application is to provide a production-ready, interactive demonstration platform showcasing data pipelines, data quality engines, and explainable AI integrations under real-world scenarios.

Key Core System Offerings:

- **Interactive Ingestion Console**: Live execution of python-based ETL pipeline jobs with step-by-step stdout log streaming.
- **Data Quality Verification**: Interactive anomalies log quarantine monitoring modeled after financial validation frameworks.
- **Orchestrated Blueprints**: Visual system flow chart showcasing skill integrations across Apache Spark, Airflow, and Snowflake.
- **Explainable AI Lab**: Attention connection graphs and SHAP contribution visualizers detailing published NASA space weather research.

2. System Architecture & Component Design

The system has been re-architected from a simple local showcase into a multi-page full-stack cloud-native application. It maintains strict separation of concerns, executing as a stateless serverless backend API and an edge-cached obfuscated static web interface.

Frontend Design System

The frontend is built using standard semantic HTML5, Vanilla CSS3, and JavaScript (ES6). Spacing is governed by HSL CSS design tokens. A responsive menu-drawer is provided for mobile viewports, stacking grid cards vertically and isolating layouts to fit standard mobile viewports (down to 320px) without horizontal page stretching.

Backend REST API

The API is written in Python utilizing the Flask framework. The backend maintains global pipeline state variables managing concurrent trigger requests and streams stdout logs to pollers via JSON API interfaces. It queries stats, breakdown summaries, and the latest quarantine logs from a SQLite database file.

SQLite Data Warehouse Schema

Data is organized into two primary SQLite tables representing clean processed analytical records and isolated, quarantined schema/bounds violations:

```
CREATE TABLE processed_metrics (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    city TEXT,
    timestamp TEXT,
    pm2_5 REAL,
    pm10 REAL,
    no2 REAL,
    o3 REAL,
    pm2_5_rolling_avg REAL,
    aqi_category TEXT,
    ingested_at TEXT
);

CREATE TABLE quarantine_records (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    city TEXT,
    timestamp TEXT,
    raw_payload TEXT,
    failure_reason TEXT,
    ingested_at TEXT
);
```

3. Pipeline & Data Quality Rules Engine

The pipeline simulates real-world distributed architectures. Upon invocation, the ETL engine handles extraction, validation, transformation, and storage sequentially:

1. Extraction

Fetches live hourly air quality reports from the Open-Meteo Air Quality API for Castro Valley, CA, Painesville, OH, and Newark, NJ. It extracts PM2.5, PM10, Nitrogen Dioxide (NO2), and Ozone (O3) parameters.

2. Ingestion & Discover DQ Rules Engine

To model Archit's real-world professional data quality experience at Discover Financial Services, all raw entries are passed through a custom Data Quality Rule Engine. To demonstrate the engine's capabilities, four simulated anomalies (negative values, null values, extreme outliers, and missing mandatory schema keys) are intentionally injected into the extraction stream. The engine evaluates each record against strict rules:

- **Mandatory Columns**: Validates presence of mandatory keys (e.g. 'city'). Violators routed to Quarantine.
- **Null Checks**: Detects missing metric readings (e.g. PM2.5 = Null). Violators routed to Quarantine.
- **Bound Constraints**: Verifies reading values are positive (e.g. PM2.5 < 0). Violators routed to Quarantine.
- **Outlier Cap**: Evaluates extreme outlier anomalies (e.g. PM2.5 > 500 ug/m3). Violators routed to Quarantine.

3. Transformation

Valid records undergo cleaning and calculation: computes 3-hour rolling averages of PM2.5 and labels EPA Air Quality Index categories (Good, Moderate, Sensitive, Unhealthy) via Pandas transformations.

4. Loading

Analytical data sets are bulk-committed to 'processed_metrics', and dirty payloads are quarantined in 'quarantine_records' alongside failure reason annotations.

4. Full-Stack Security Hardening & Obfuscation

To ensure the website remains secure and resilient in a public environment, robust defense measures have been applied to both the client-side files and backend APIs.

Client-Side Inspect Element Protection

To deter unauthorized copying, a dedicated script (security.js) blocks common inspection shortcuts. Furthermore, if DevTools is opened, an active debugger trap pauses browser execution, rendering the inspector frozen.

Automated Obfuscation Build Pipeline

We implemented an automated build script (build.js) that compiles raw assets from 'frontend/' into 'dist/' using DevDependencies. It minifies HTML and CSS, and obfuscates JavaScript using javascript-obfuscator:

- **Self-Defending Lock**: Script crashes automatically if pretty-printed or formatted.
- **Control Flow Flattening**: Obfuscates code logical structures, making it unreadable.
- **Silencing Outputs**: Removes all diagnostic console.log calls in production.

Backend API Hardening

The Flask backend has been hardened against attack vectors: debug mode is disabled by default, preventing arbitrary code execution via Flask tracebacks. Additionally, CORS is restricted to allow connections only from the frontend's domain, and local bindings are isolated during development.

5. Google Cloud Platform (GCP) Deployment

The project is hosted in production on Google Cloud Platform, leveraging serverless components to stay entirely within GCP's free tiers.

Frontend: Firebase Hosting CDN

The obfuscated 'dist/' directory is hosted on Firebase Hosting. Custom HTTP security headers are enforced via firebase.json to block framing, prevent MIME-sniffing, restrict permissions, and enforce a strict Content Security Policy:

```
"headers": [
  { "key": "X-Frame-Options", "value": "DENY" },
  { "key": "X-Content-Type-Options", "value": "nosniff" },
  { "key": "Content-Security-Policy", "value": "default-src 'self'; script-src 'self'
    https://cdn.jsdelivr.net; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com
    https://cdnjs.cloudflare.com; connect-src 'self' http://127.0.0.1:5000
    https://archit-de-portfolio-8902.uc.r.appspot.com https://air-quality-api.open-meteo.com;" }
]
```

Backend: Google App Engine Standard

The Flask API backend is deployed on App Engine Standard. It runs on a free F1 instance using the Python 3.12 runtime and Gunicorn. It connects securely to the SQLite database and handles requests on demand.

6. Project Directory Map

Below is a directory mapping of the core files in the repository, outlining their roles in development and deployment:

File / Folder	Target Environment	Function / Role
run.bat	Local Windows Dev	Launcher; sets dev environment and launches Flask/HTTP servers.
package.json	Build Environment	Defines npm tasks and build dependencies for minifiers & obfuscators.
build.js	Build Node Runtime	Automated compiler script mapping frontend/ to protected dist/.
firebase.json	Firebase Hosting CDN	Defines static root and embeds HTTP security headers (CSP, Frame options).
netlify.toml	Netlify Cloud CDN	Secondary configuration mapping build target dist/ and headers.

backend/app.py	Flask App Engine API	REST endpoint handler; queries SQLite statistics and handles ETL triggers.
backend/pipeline.py	Python Backend	ETL process engine featuring the Custom Discover DQ rules checks.
backend/app.yaml	App Engine Standard	Declares python312 runtime, F1 class, and CORS Allowed Origins.
frontend/security.js	Obfuscated Frontend	Active protection scripts blocking mouse context and key shortcuts.